

FarmOS Domain 01 API Integration Guide

Identity, authentication, organization, roles, assignments, access governance, and audit | API v2.2

Scope: this guide is strictly limited to the locked Domain 01 correction. Animal, Health, and Operations APIs are intentionally excluded and are not reviewed or approved by this document.

Backend authorization remains authoritative; the frontend must not infer permission from role names.

1. Integration sequence

Follow this order after the user opens the application:

1. Send the channel header. 2. Log in. 3. Store cookies automatically. 4. Fetch the current user profile. 5. Fetch capabilities and navigation. 6. Render only permitted governance screens. 7. Call organization/farm endpoints with authorized identifiers. 8. Handle structured error codes.

Required headers

```
{
  "Content-Type": "application/json",
  "X-App-Channel": "web",
  "X-CSRFToken": "value-from-client_csrf_token"
}
```

Use X-App-Channel: mobile for the mobile client. Do not store access or refresh tokens in browser localStorage; the backend uses secure cookies.

2. Standard response envelopes

Every v2 endpoint returns a predictable JSON envelope. Read data from data and pagination information from meta.

```
{
  "success": true,
  "code": "REQUEST_SUCCESSFUL",
  "message": "Request completed successfully.",
  "data": {
    "id": 123
  },
  "meta": null
}
{
  "success": false,
  "code": "PERMISSION_DENIED",
  "message": "You do not have permission to perform this action.",
  "data": null,
  "errors": {},
  "retryable": false
}
```

3. Authentication and bootstrap

POST

/api/auth/login

Purpose: Authenticate an active user and create the secure Web/Mobile session.

Request JSON

```
{
  "email": "owner@example.com",
  "password": "your-password"
}
```

Response JSON

```
{
  "status": "Success",
  "message": "Login successful",
  "is_admin": false
}
```

Frontend notes: The response sets client_access_token, client_refresh_token, and client_csrf_token cookies. A deactivated account returns ACCOUNT_DEACTIVATED.

GET

/api/v2/users/me/

Purpose: Load the signed-in user identity, organization, ownership, assignments, and channel access.

Response JSON

```
{
  "success": true,
  "code": "REQUEST_SUCCESSFUL",
  "data": {
    "identity": {
      "id": "uuid",
      "display_name": "Ada Farmer",
      "email": "owner@example.com"
    },
    "access": {
      "account_type": "organization_owner",
      "access_source": "organization_ownership",
      "scope": "organization",
      "is_organization_owner": true,
      "all_farms": true,
      "assignments": []
    },
    "channel_access": {
      "web": true,
      "mobile": false
    }
  }
}
```

Frontend notes: Use this response to build the account shell. Owner status is a boolean/context decision, not a role-name comparison.

GET

/api/v2/users/me/capabilities/

Purpose: Load effective capabilities and navigation primitives.

Response JSON

```
{
  "success": true,
  "data": {
    "permissions": [
      "view_animal_details",
      "view_operation"
    ],
    "capabilities": {
      "view_animal_details": true,
      "create_operation": false
    },
    "navigation": {
      "livestock": true,
      "operations": true,

```

```

    "people": false
  },
  "channel_access": {
    "web": true,
    "mobile": true
  }
}
}

```

Frontend notes: Render actions from capabilities. Do not assume Farm Manager or Veterinarian authority from labels.

POST `/api/auth/refresh-token`

Purpose: Renew an expired access session using the refresh cookie.

Response JSON

```

{
  "success": true,
  "message": "Token refreshed"
}

```

Frontend notes: If renewal fails, redirect to login. Deactivation cannot be bypassed through refresh.

POST `/api/auth/signout`

Purpose: Terminate the current authenticated session.

Response JSON

```

{
  "success": true,
  "message": "Logged out successfully"
}

```

Frontend notes: After logout, protected requests must fail; the account remains eligible for a fresh login.

4. Organization and people

GET `/api/v2/users/?page=1&page_size=20`

Purpose: List members of the authenticated organization.

Response JSON

```

{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "display_name": "Ada Farmer",
      "email": "owner@example.com",
      "account_status": "active",
      "owner": true
    }
  ],
  "meta": {
    "pagination": {
      "page": 1,
      "page_size": 20,
      "total_items": 1,
      "total_pages": 1
    }
  }
}

```

Frontend notes: The owner field is a boolean. Search with ?search=. Never filter another organization in the frontend.

PATCH `/api/v2/users/me/`

Purpose: Update permitted personal profile fields only.

Request JSON

```
{
  "display_name": "Ada Farmer",
  "phone": "08012345678"
}
```

Response JSON

```
{
  "success": true,
  "code": "PROFILE_UPDATED",
  "message": "Profile updated successfully.",
  "data": {
    "user": {
      "display_name": "Ada Farmer",
      "phone": "08012345678"
    }
  }
}
```

Frontend notes: Any authenticated active user may update their own display_name and phone. Do not send role, permissions, farm_assignment, account_status, or organization_ownership; those fields are rejected.

POST `/api/v2/users/me/avatar/`

Purpose: Upload or replace the current user's profile picture using multipart/form-data.

Request JSON

"multipart/form-data: avatar=@profile.jpg (JPG or PNG, maximum 5 MB)"

Response JSON

```
{
  "success": true,
  "code": "PROFILE_IMAGE_UPDATED",
  "message": "Profile picture updated successfully.",
  "data": {
    "avatar_url": "https://api.example.com/media/avatars/profile.jpg"
  }
}
```

Frontend notes: Any authenticated active user may manage their own picture. Send the file as avatar; do not send JSON for this request.

DELETE `/api/v2/users/me/avatar/`

Purpose: Remove the current user's profile picture without deleting the account or profile.

Response JSON

```
{
  "success": true,
  "code": "PROFILE_IMAGE_REMOVED",
  "message": "Profile picture removed successfully.",
  "data": {
    "avatar_url": null
  }
}
```

Frontend notes: The operation affects only the signed-in user's avatar. Deactivated users cannot call protected profile endpoints.

GET `/api/v2/organizations/{organization_id}/`

Purpose: Read the organization profile and ownership context.

Response JSON

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "name": "Green Valley Farm",
    "country": "Nigeria",
    "state_region": "Lagos",
    "owner": {
      "id": "uuid",
      "is_owner": true
    }
  }
}
```

```

    }
  }
}

```

5. Account-state lifecycle

Account state is evaluated on login, protected requests, and refresh. The lifecycle is ACTIVE → DEACTIVATED → REACTIVATED. Deactivation preserves profile, assignment history, and audit history; it does not delete the user.

```

{
  "account_status": "active"
}

```

POST `/api/v2/users/{user_id}/deactivate/`

Purpose: Deactivate a staff account from an authorized governance context.

Request JSON

```

{
  "reason": "No longer employed"
}

```

Response JSON

```

{
  "success": true,
  "data": {
    "user_id": "uuid",
    "account_status": "deactivated"
  }
}

```

Frontend notes: Existing sessions fail on the next protected request. Fresh login and refresh return ACCOUNT_DEACTIVATED.

POST `/api/v2/users/{user_id}/reactivate/`

Purpose: Restore account eligibility without recreating revoked assignments.

Response JSON

```

{
  "success": true,
  "data": {
    "user_id": "uuid",
    "account_status": "active"
  }
}

```

Frontend notes: The user may log in again; effective access is recalculated from current membership and assignments.

6. Audit and security events

The backend emits durable security/access events. The frontend should display structured event fields and must never parse a generic summary string.

```

{
  "event_name": "USER_ASSIGNMENT_UPDATED",
  "actor_id": "uuid",
  "target_id": "assignment-id",
  "organization_id": "uuid",
  "farm_id": 1,
  "timestamp": "2026-09-16T08:00:00Z",
  "previous_state": {
    "farm_id": 1
  },
  "new_state": {
    "farm_id": 2
  },
  "result": "success",
  "source": "api"
}

```

Implemented event names include USER_LOGGED_IN, USER_LOGGED_OUT, USER_DEACTIVATED, USER_REACTIVATED, USER_ASSIGNMENT_CREATED, USER_ASSIGNMENT_UPDATED,

USER_ASSIGNMENT_REVOKED, ROLE_CREATED, ROLE_UPDATED, ROLE_PERMISSIONS_UPDATED, ORGANIZATION_CREATED, and FARM_CREATED.

Evidence: the development test suite verifies login, assignment, deactivation/reactivation, audit context, timestamps, result/source fields, and exclusion of passwords, access tokens, refresh tokens, and CSRF secrets.

7. Organization farms

GET

/api/v2/organizations/{organization_id}/farms/

Purpose: List farms visible to the current user.

Response JSON

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "North Farm",
      "farm_code": "FRM-001",
      "status": "active"
    }
  ],
  "meta": {
    "pagination": {
      "page": 1,
      "page_size": 20,
      "total_items": 1
    }
  }
}
```

Frontend notes: Use returned farm IDs for subsequent scoped requests.

GET

/api/v2/farms/{farm_id}/

Purpose: Read a farm profile and scoped counts.

Response JSON

```
{
  "success": true,
  "data": {
    "id": 1,
    "organization_id": "uuid",
    "name": "North Farm",
    "status": "active",
    "counts": {
      "animals": 42,
      "open_tasks": 3
    }
  }
}
```

8. Roles, assignments, and channel rules

Role and assignment administration is available to owners or users with the relevant governance capability.

```
{
  "staff_role_templates": {
    "Farm Manager": {
      "web": true,
      "mobile": true
    },
    "Veterinarian": {
      "web": true,
      "mobile": true
    },
    "Field Worker": {
      "web": false,
      "mobile": true
    }
  }
}
```

Organization Owner is deliberately absent from role_templates: Owner authority comes from organization ownership, not a staff role.

Assignment payload:

```
{
  "user_id": "uuid",
```

```

    "role_id": 3,
    "farm_id": 1,
    "client_request_id": "assignment-001"
  }

```

A revoked assignment remains in history but no longer grants current farm authority. A Field Worker calling a Web-protected endpoint receives CHANNEL_ACCESS_DENIED.

9. Error handling and retry behavior

```

{
  "success": false,
  "code": "FARM_ACCESS_DENIED",
  "message": "You do not have access to this farm.",
  "data": null,
  "errors": {},
  "retryable": false
}

```

Handle codes, not message text. Recommended frontend actions:

AUTHENTICATION_REQUIRED / SESSION_EXPIRED → refresh once or redirect to login. ACCOUNT_DEACTIVATED → sign out and show account notice. PERMISSION_DENIED / FARM_ACCESS_DENIED / CHANNEL_ACCESS_DENIED → show access-denied state. VALIDATION_ERROR → show field errors. CONFLICT → preserve the server result or ask the user to resolve the conflict.

10. Implementation evidence

The development Docker verification currently reports 40 automated tests passed and 27 account/API end-to-end checks passed. These tests cover ownership, bootstrap, role templates, assignments, channels, deactivation/reactivation, refresh, logout, isolation, audit context, stable errors, soft removal, and sequential idempotency. Concurrent duplicate requests remain a separate QA item.

11. Implementation checklist

✓ Send X-App-Channel. ✓ Use secure cookies. ✓ Bootstrap with /users/me/ and /users/me/capabilities/. ✓ Use capability booleans for UI actions. ✓ Include farm_id on scoped writes. ✓ Include client_request_id on retryable material writes. ✓ Read data from the standard envelope. ✓ Never parse exception text. ✓ Never allow frontend-only tenant filtering. ✓ Use multipart/form-data for avatar uploads; enforce JPG/PNG and 5 MB maximum.

Contract reference: FarmOS Frontend API Contract v2.2. Domain 01 scope only. Backend base URL for Docker development: <http://127.0.0.1:8082>.